MODUL PRAKTIKUM

# Autentikasi dan Otorisasi dengan JWT pada REST API

*Pertemuan 7 — Login, token, role, rate limiting, dan validasi input*

Mata Kuliah Teknologi Web Service

Program Studi Informatika • 2 SKS • Semester Gasal 2026/2027

# Daftar Isi


- 1. Tujuan Praktikum
- 2. Alat dan Bahan
- 3. Konsep Singkat dan Peta Perubahan dari Pertemuan 6
- 4. Langkah 1–3: Database dan Latihan JWT di Terminal
- 5. Langkah 4–8: Menambah Autentikasi ke Server
- 6. Langkah 9–10: Menguji dengan Postman dan Client Otomatis
- 7. Langkah 11: Client Web dengan Login
- 8. Langkah 12: Rate Limiting dan Validasi Input
- 9. Bacaan: Pengantar OAuth2
- 10. Pemecahan Masalah
- 11. Lembar Observasi
- 12. Ide Tugas
- 13. Referensi

# 1. Tujuan Praktikum

Setelah praktikum ini, mahasiswa mampu:

- Membedakan **autentikasi** (siapa kamu?) dan **otorisasi** (bolehkah kamu?) dan menunjukkan perbedaannya lewat status **401** dan **403**.
- Menyimpan password dengan **hash** (`password_hash`) dan memeriksanya (`password_verify`).
- Membuat endpoint **login** yang menghasilkan token **JWT**, dan menjelaskan isi serta cara kerja tanda tangannya.
- Melindungi endpoint dengan token **Bearer** dan membatasi aksi berdasarkan **role** (admin/user).
- Menerapkan praktik keamanan dasar: **validasi input**, **rate limiting**, prepared statement, pesan error yang tidak membocorkan.
- Mencatat siapa mengakses apa pada **log di database**.

## 2. Alat dan Bahan

- Proyek `api-mhs-auth` (pengembangan dari `api-mhs-sederhana` Pertemuan 6) — dari dosen.
- XAMPP/Laragon (Apache + MySQL + phpMyAdmin), PHP 8.x, Postman, VS Code, browser.

> **Prasyarat**
> Praktikum Pertemuan 5–6 (API sederhana + logging di database) sudah dipahami. Pada modul ini kode yang **baru** ditandai komentar `<-- BARU P7`.

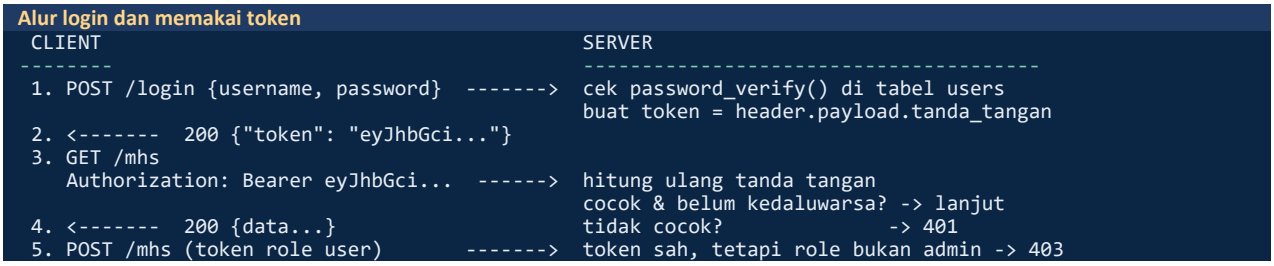
# 3. Konsep Singkat dan Peta Perubahan

## Autentikasi vs otorisasi

| | Autentikasi (Authentication) | Otorisasi (Authorization) |
|---|---|---|
| Pertanyaan | Siapa kamu? | Apa yang boleh kamu lakukan? |
| Dibuktikan dengan | Username + password, token | Role / izin yang melekat pada akun |
| Urutan | Terjadi lebih dulu | Terjadi setelah identitas diketahui |
| Gagal → status | **401** Unauthorized | **403** Forbidden |
| Pada proyek ini | `wajib_login()` | `wajib_admin()` |

## Alur kerja JWT

**Alur login dan memakai token**




## Peta hak akses endpoint

| Method | URL | Akses | Hasil jika tidak berhak |
|---|---|---|---|
| POST | /login | Publik | 401 salah \| 422 data kurang \| 429 terlalu sering salah |
| GET | /me | Login | 401 tanpa token |
| GET | /mhs, /mhs/{id} | Login | 401 tanpa token |
| POST/PUT/DELETE | /mhs, /mhs/{id} | Admin | 401 tanpa token \| **403** jika role user |
| GET | /log | Admin | 401 \| 403 |

## Apa yang berubah dari Pertemuan 6?

| File | Perubahan |
|---|---|
| `database/mhs.sql` | Tabel baru `users` (password ter-hash, kolom role); kolom `username` pada `log_api`. |
| `server/jwt.php` (BARU) | `buat_token()` dan `cek_token()`. |
| `server/koneksi.php` | Tambahan: `$jwt_secret`, `$jwt_berlaku`, `$batas_login`, `$batas_umum`. |
| `server/fungsi.php` | Tambahan: `ambil_token()`, `wajib_login()`, `wajib_admin()`, `batasi_request()`; log mencatat username. |
| `server/index.php` | Endpoint `/login` dan `/me`; semua endpoint lain wajib token; `wajib_admin()` pada POST/PUT/DELETE dan /log. |
| `server/.htaccess` | Meneruskan header `Authorization` ke PHP. |
| `client/*` | Client mengirim header `Authorization: Bearer ...`; ada form login. |
| `latihan/` (BARU) | `jwt_demo.php` dan `uji_rate_limit.php` untuk belajar. |

# 4. Langkah 1–3: Database dan Latihan JWT

## LANGKAH 1 Menyiapkan proyek dan database baru

1. Ekstrak `api-mhs-auth.zip` dan salin folder `api-mhs-auth` ke `C:\xampp\htdocs\`. Server: `http://localhost/api-mhs-auth/server`.
2. phpMyAdmin → tab **Import** → pilih `database/mhs.sql` → **Go**.
3. Periksa tabel `mhs` (3 baris), `users` (2 baris), `log_api` (kosong).

> **Perhatian**
>
> `mhs.sql` memakai `DROP TABLE`, sehingga data lama di `kampus_db` akan terhapus. Jika ingin menyimpan data Pertemuan 6, gunakan nama database lain dan ubah `$nama_db` di `koneksi.php`.

## LANGKAH 2 Memahami tabel users dan hash password

Perhatikan kolom `password`: isinya bukan `admin123`, melainkan **hash** acak. Hash tidak dapat dikembalikan menjadi password asli. Jika database bocor, penyerang tidak langsung mendapatkan password pengguna.

**database/mhs.sql (tabel users dan akun latihan)**

```
-- TABEL 2: users  -> akun yang boleh login (BARU di Pertemuan 7)
--   password disimpan dalam bentuk HASH, BUKAN teks asli.
--   role menentukan hak akses (otorisasi): 'admin' atau 'user'.
-- ------------------------------------------------------------------
DROP TABLE IF EXISTS users;
CREATE TABLE users (
  id        INT AUTO_INCREMENT PRIMARY KEY,
  username  VARCHAR(50)  NOT NULL UNIQUE,
  password  VARCHAR(255) NOT NULL,                  -- hasil password_hash()
  role      VARCHAR(10)  NOT NULL DEFAULT 'user'    -- 'admin' atau 'user'
);

-- ------------------------------------------------------------------
```

| Username | Password | Role | Keterangan |
|---|---|---|---|
| admin | admin123 | admin | Boleh CRUD dan melihat log |
| budi | budi123 | user | Hanya boleh melihat data |

## LANGKAH 3 Latihan JWT di terminal — jwt_demo.php

Sebelum membaca kode server, lihat dulu apa itu JWT. Skrip ini membuat token, membuka isinya, lalu membuktikan bahwa token yang diubah atau kedaluwarsa ditolak.

**Terminal VS Code (dari folder proyek)**

```
php latihan/jwt_demo.php
```

**Contoh keluaran**

```
1) MEMBUAT TOKEN

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsInVzZXJuYW1lIjoiYnVkaSIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzk...

2) ISI TOKEN BISA DIBACA SIAPA SAJA (hanya di-encode, tidak dienkripsi)
   header  : {"alg":"HS256","typ":"JWT"}
   payload : {"sub":1,"username":"budi","role":"user","iat":1792538476,"exp":1792542076}
   tanda tangan (tidak bisa dibaca): R3xb9Mkp2iYucl_dLHLXOLEsLcgJ3u34Rs7WaKVzr5g

3) MEMERIKSA TOKEN YANG ASLI
   SAH, username = budi, role = user

4) PENYERANG MENGUBAH ROLE MENJADI ADMIN (tanpa tahu kunci rahasia)
   DITOLAK -> tanda tangan tidak cocok dengan isi yang baru

5) TOKEN DENGAN KUNCI RAHASIA YANG BERBEDA
   DITOLAK
```

```
6) TOKEN KEDALUWARSA (masa berlaku -10 detik)
   DITOLAK -> sudah lewat waktu exp

7) HASH PASSWORD
   hash        : $2y$10$PWl/PZXL9z1a9jlI6T2Q5uTZXeWT4VXbMp23LnA5dEYnwOrGmoML2
   cocok 'budi123' ? ya
   cocok 'salah'   ? tidak
   (hash yang sama TIDAK bisa dikembalikan menjadi password asli)
```

**Yang harus diingat**

Payload JWT hanya **di-encode (base64)**, bukan dienkripsi. Siapa pun dapat membacanya (coba tempel token di jwt.io). Karena itu **jangan menaruh password atau data rahasia di payload**. Keamanan JWT berasal dari **tanda tangan**: isi tidak dapat diubah tanpa kunci rahasia server.

# 5. Langkah 4–8: Menambah Autentikasi ke Server

## LANGKAH 4 jwt.php — membuat dan memeriksa token

Dua fungsi inti. `buat_token()` menyusun header, payload (ditambah `iat` dan `exp`), lalu menandatanganinya dengan HMAC-SHA256. `cek_token()` menghitung ulang tanda tangan dan membandingkannya, lalu memeriksa masa berlaku.

**server/jwt.php**

```php
<?php
// ======================================================================
// jwt.php  --  Membuat dan memeriksa JSON Web Token (JWT)
//
// JWT adalah teks yang terdiri dari 3 bagian dipisah titik:
//
//      HEADER . PAYLOAD . SIGNATURE
//
//   HEADER    = info algoritma          {"alg":"HS256","typ":"JWT"}
//   PAYLOAD   = isi data (klaim)        {"sub":1,"username":"admin","role":"admin","exp":...}
//   SIGNATURE = tanda tangan digital    HMAC-SHA256(header.payload, KUNCI RAHASIA)
//
// PENTING: header dan payload HANYA di-encode (bisa dibaca siapa saja), TIDAK dienkripsi.
//          Jangan taruh password atau data rahasia di payload!
//          Keamanan datang dari SIGNATURE: jika isi diubah, tanda tangan tidak cocok lagi.
// ======================================================================

// Base64 versi aman untuk URL: ganti + dan / menjadi - dan _, buang tanda =.
function base64url_encode($teks)
{
    return rtrim(strtr(base64_encode($teks), '+/', '-_'), '=');
}

function base64url_decode($teks)
{
    return base64_decode(strtr($teks, '-_', '+/'));
}

// ----------------------------------------------------------------------
// buat_token() : membuat JWT dari data payload
//   $payload = array data, mis. ['sub' => 1, 'username' => 'admin', 'role' => 'admin']
//   $rahasia = kunci rahasia server
//   $detik   = masa berlaku token
// ----------------------------------------------------------------------
function buat_token($payload, $rahasia, $detik)
{
    $payload['iat'] = time();            // iat = issued at (kapan dibuat)
    $payload['exp'] = time() + $detik;   // exp = expired (kapan kedaluwarsa)

    $header = base64url_encode(json_encode(['alg' => 'HS256', 'typ' => 'JWT']));
    $isi    = base64url_encode(json_encode($payload));

    // Tanda tangan = HMAC-SHA256 dari "header.isi" memakai kunci rahasia.
    $tanda_tangan = base64url_encode(hash_hmac('sha256', "$header.$isi", $rahasia, true));

    return "$header.$isi.$tanda_tangan";
}

// ----------------------------------------------------------------------
// cek_token() : memeriksa token
//   Mengembalikan array payload jika token SAH, atau null jika tidak sah.
// ----------------------------------------------------------------------
function cek_token($token, $rahasia)
{
    // 1) Harus terdiri dari tepat 3 bagian.
    $bagian = explode('.', $token);
    if (count($bagian) != 3) {
        return null;
    }
    list($header, $isi, $tanda_tangan) = $bagian;

    // 2) Hitung ulang tanda tangan yang SEHARUSNYA, lalu bandingkan.
    //    Kita SELALU memakai HS256 (tidak mempercayai "alg" dari token) -> mencegah serangan "alg: none".
    //    hash_equals = perbandingan aman dari serangan pengukuran waktu.
    $seharusnya = base64url_encode(hash_hmac('sha256', "$header.$isi", $rahasia, true));
    if (!hash_equals($seharusnya, $tanda_tangan)) {
        return null;   // isi pernah diubah, atau ditandatangani dengan kunci lain
    }
```

```
    // 3) Buka isi payload.
    $payload = json_decode(base64url_decode($isi), true);
    if (!is_array($payload)) {
        return null;
    }

    // 4) Periksa masa berlaku.
    if (!isset($payload['exp']) || $payload['exp'] < time()) {
        return null;   // sudah kedaluwarsa
    }

    return $payload;
}
```

## LANGKAH 5 koneksi.php — pengaturan keamanan

server/koneksi.php

```
// ---------------- Pengaturan keamanan (BARU di Pertemuan 7) ----------------

// Kunci rahasia untuk "menandatangani" token JWT.
// Siapa pun yang tahu kunci ini bisa membuat token palsu, jadi:
//   - pilih teks panjang dan acak,
//   - di aplikasi sungguhan simpan di environment variable, jangan di kode.
$jwt_secret = 'ganti-dengan-teks-rahasia-yang-panjang-dan-acak-123';

// Token berlaku berapa detik? 3600 = 1 jam.
$jwt_berlaku = 3600;

// RATE LIMITING (pembatasan jumlah request):
$batas_login = 5;     // maksimal 5x salah login per username per menit
$batas_umum  = 120;   // maksimal 120 request per IP per menit
```

> **Kunci rahasia**
>
> Ganti `$jwt_secret` dengan teks acak yang panjang. Jika kunci diubah, **semua token lama menjadi tidak sah** (pengguna harus login ulang). Pada aplikasi sungguhan kunci disimpan di environment variable, bukan di kode, dan tidak boleh masuk repository publik.

## LANGKAH 6 fungsi.php — wajib_login(), wajib_admin(), rate limiting

Perhatikan pemisahan tugas: `wajib_login()` = **autentikasi** (401), `wajib_admin()` = **otorisasi** (403). Fungsi `catat_log()` kini menyimpan juga **username**.

server/fungsi.php

```
<?php
// ======================================================================
// fungsi.php  --  Fungsi bantu: log, kirim jawaban, keamanan
// ======================================================================

// Nama pengguna yang sedang mengakses. Diisi saat login / token diperiksa,
// lalu ikut dicatat di log supaya kita tahu "siapa melakukan apa".
$username_log = null;

// ----------------------------------------------------------------------
// catat_log() : menyimpan satu baris ke tabel log_api (LOGGING DI DATABASE)
// ----------------------------------------------------------------------
function catat_log($status, $pesan = '')
{
    global $db, $username_log;

    $waktu  = date('Y-m-d H:i:s');
    $method = $_SERVER['REQUEST_METHOD'];
    $path   = $_SERVER['REQUEST_URI'];
    $ip     = $_SERVER['REMOTE_ADDR'];

    $sql  = "INSERT INTO log_api (waktu, method, path, status, ip, username, pesan)
             VALUES (?, ?, ?, ?, ?, ?, ?)";
    $stmt = $db->prepare($sql);
    $stmt->execute([$waktu, $method, $path, $status, $ip, $username_log, $pesan]);
}
```

```php
// ---------------------------------------------------------------------
// kirim() : mengirim jawaban JSON, mencatat log, lalu berhenti
// ---------------------------------------------------------------------
function kirim($data, $status = 200)
{
    $pesan = isset($data['pesan']) ? $data['pesan'] : 'OK';
    catat_log($status, $pesan);

    http_response_code($status);
    header('Content-Type: application/json');
    echo json_encode($data, JSON_PRETTY_PRINT);
    exit;
}

// ---------------------------------------------------------------------
// ambil_token() : membaca token dari header
//     Authorization: Bearer eyJhbGciOi...
// Mengembalikan teks token, atau null jika header tidak ada / bentuknya salah.
// ---------------------------------------------------------------------
function ambil_token()
{
    $header = '';
    if (isset($_SERVER['HTTP_AUTHORIZATION'])) {
        $header = $_SERVER['HTTP_AUTHORIZATION'];
    } elseif (isset($_SERVER['REDIRECT_HTTP_AUTHORIZATION'])) {
        $header = $_SERVER['REDIRECT_HTTP_AUTHORIZATION'];     // dipakai pada beberapa konfigurasi Apache
    }

    // Bentuk yang benar: "Bearer <token>"
    if (preg_match('/^Bearer\s+(\S+)$/', $header, $cocok)) {
        return $cocok[1];
    }
    return null;
}

// ---------------------------------------------------------------------
// wajib_login() : AUTENTIKASI -> "Siapa kamu?"
//   Memeriksa token. Jika tidak sah -> 401 dan berhenti.
//   Jika sah, mengembalikan data pengguna dari token.
// ---------------------------------------------------------------------
function wajib_login()
{
    global $jwt_secret, $username_log;

    $token   = ambil_token();
    $payload = $token ? cek_token($token, $jwt_secret) : null;

    if (!$payload) {
        header('WWW-Authenticate: Bearer');   // petunjuk standar: "pakai token Bearer"
        kirim(['status' => 'error', 'pesan' => 'Token tidak ada, tidak valid, atau sudah kedaluwarsa'],
401);
    }

    $username_log = $payload['username'];     // supaya tercatat di log
    return $payload;
}

// ---------------------------------------------------------------------
// wajib_admin() : OTORISASI -> "Bolehkah kamu melakukan ini?"
//   Pengguna sudah dikenali (login), tetapi rolenya harus 'admin'.
//   Jika bukan -> 403 Forbidden.
// ---------------------------------------------------------------------
function wajib_admin($pengguna)
{
    if ($pengguna['role'] != 'admin') {
        kirim(['status' => 'error', 'pesan' => 'Akses ditolak: hanya admin yang boleh'], 403);
    }
}

// ---------------------------------------------------------------------
// batasi_request() : RATE LIMITING umum per IP
//   Menghitung jumlah request dari IP ini dalam 1 menit terakhir (dari tabel log_api).
//   Jika sudah terlalu banyak -> 429 Too Many Requests.
// ---------------------------------------------------------------------
function batasi_request()
{
    global $db, $batas_umum;

    $semenit_lalu = date('Y-m-d H:i:s', time() - 60);
```

```
    $stmt = $db->prepare("SELECT COUNT(*) FROM log_api WHERE ip = ? AND waktu >= ?");
    $stmt->execute([$_SERVER['REMOTE_ADDR'], $semenit_lalu]);
    $jumlah = $stmt->fetchColumn();

    if ($jumlah >= $batas_umum) {
        header('Retry-After: 60');           // beri tahu client: coba lagi 60 detik lagi
        kirim(['status' => 'error', 'pesan' => 'Terlalu banyak request. Coba lagi 1 menit lagi'], 429);
    }
}
```

## LANGKAH 7 index.php — login, token wajib, dan role

Baca bagian bertanda `<-- BARU P7`. Urutan penting: (1) `batasi_request()`, (2) `/login` yang publik, (3) `wajib_login()` untuk semua endpoint lain, (4) `wajib_admin()` pada aksi yang mengubah data.

server/index.php

```
<?php
// ======================================================================
// index.php  --  API Mahasiswa (versi sederhana)
//
// Semua alamat masuk ke file ini berkat .htaccess. Alurnya:
//   1. Baca alamat (URL) dan jenis request (GET/POST/PUT/DELETE)
//   2. Tentukan mau ke mana (routing)
//   3. Jalankan query database
//   4. Kirim jawaban JSON  (+ catat ke tabel log_api)
//
// Daftar endpoint:
//   POST   /login      -> login, mendapat token JWT          (publik)
//   GET    /me         -> lihat data akun yang sedang login  (perlu token)
//   GET    /mhs        -> ambil semua mahasiswa              (perlu token)
//   GET    /mhs/3      -> ambil mahasiswa dengan id 3        (perlu token)
//   POST   /mhs        -> tambah mahasiswa baru              (token + role admin)
//   PUT    /mhs/3      -> ubah data mahasiswa id 3           (token + role admin)
//   DELETE /mhs/3      -> hapus mahasiswa id 3               (token + role admin)
//   GET    /log        -> lihat 20 catatan log terakhir      (token + role admin)
//
// Perubahan dari Pertemuan 6 ditandai komentar  "<-- BARU P7"
// ======================================================================

require 'koneksi.php';   // menghasilkan variabel $db (koneksi database) + pengaturan keamanan
require 'jwt.php';       // <-- BARU P7: fungsi buat_token() dan cek_token()
require 'fungsi.php';    // fungsi kirim(), catat_log(), wajib_login(), wajib_admin(), ...

// ----------------------------------------------------------------------
// Izin untuk client web (browser) yang alamatnya berbeda dari server ini.
// Tanpa ini browser akan memblokir permintaan (aturan CORS).
// ----------------------------------------------------------------------
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');   // <-- BARU P7: izinkan header
Authorization

// Browser kadang bertanya dulu "boleh tidak?" dengan method OPTIONS. Jawab saja "boleh".
if ($_SERVER['REQUEST_METHOD'] == 'OPTIONS') {
    http_response_code(204);
    exit;
}

// ----------------------------------------------------------------------
// LANGKAH 1: membaca alamat
// Contoh alamat: /api-mhs-sederhana/server/mhs/3
// Kita buang folder server-nya, sisanya "mhs/3" dipecah jadi ['mhs', '3'].
// ----------------------------------------------------------------------
$path   = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);   // tanpa ?query
$folder = dirname($_SERVER['SCRIPT_NAME']);                   // folder tempat index.php
if ($folder != '/' && strpos($path, $folder) === 0) {
    $path = substr($path, strlen($folder));                   // buang nama folder
}
$bagian   = explode('/', trim($path, '/'));                   // pecah per tanda "/"
$resource = $bagian[0];                                       // 'mhs' atau 'log'
$id       = isset($bagian[1]) ? $bagian[1] : null;            // '3' (boleh kosong)

$method = $_SERVER['REQUEST_METHOD'];                         // GET / POST / PUT / DELETE

// Data JSON yang dikirim client (untuk POST dan PUT), diubah menjadi array PHP.
$input = json_decode(file_get_contents('php://input'), true);
```

```php
// <-- BARU P7: RATE LIMITING umum. Terlalu banyak request dari satu IP -> 429.
batasi_request();

// try ... catch : jika ada error database, jangan tampilkan layar error PHP,
// tapi kirim jawaban JSON yang rapi.
try {

    // ================================================================
    // <-- BARU P7: POST /login  -> AUTENTIKASI (publik, tidak perlu token)
    // ================================================================
    if ($resource == 'login' && $method == 'POST') {

        // 1) Validasi input: username dan password wajib diisi (teks).
        if (!is_array($input) || empty($input['username']) || empty($input['password'])
            || !is_string($input['username']) || !is_string($input['password'])) {
            kirim(['status' => 'error', 'pesan' => 'username dan password wajib diisi'], 422);
        }
        $username = trim($input['username']);
        $username_log = $username;                      // ikut dicatat di log

        // 2) RATE LIMITING login: hitung gagal login untuk username ini dari IP ini dalam 1 menit.
        $semenit_lalu = date('Y-m-d H:i:s', time() - 60);
        $stmt = $db->prepare("SELECT COUNT(*) FROM log_api
                              WHERE ip = ? AND username = ? AND status = 401 AND waktu >= ?
                                AND path LIKE '%/login'");
        $stmt->execute([$_SERVER['REMOTE_ADDR'], $username, $semenit_lalu]);
        if ($stmt->fetchColumn() >= $batas_login) {
            header('Retry-After: 60');
            kirim(['status' => 'error', 'pesan' => 'Terlalu banyak percobaan login. Coba lagi 1 menit
lagi'], 429);
        }

        // 3) Cari akun di database (prepared statement -> aman dari SQL Injection).
        $stmt = $db->prepare("SELECT id, username, password, role FROM users WHERE username = ?");
        $stmt->execute([$username]);
        $akun = $stmt->fetch();

        // 4) Cocokkan password. password_verify membandingkan password dengan HASH di database.
        //    Pesan error SENGAJA sama untuk "username salah" dan "password salah",
        //    supaya penyerang tidak tahu username mana yang benar-benar ada.
        if (!$akun || !password_verify($input['password'], $akun['password'])) {
            kirim(['status' => 'error', 'pesan' => 'Username atau password salah'], 401);
        }

        // 5) Berhasil -> buat token JWT berisi identitas pengguna (BUKAN password!).
        $token = buat_token(
            ['sub' => (int)$akun['id'], 'username' => $akun['username'], 'role' => $akun['role']],
            $jwt_secret,
            $jwt_berlaku
        );
        kirim([
            'status'        => 'sukses',
            'pesan'         => 'Login berhasil',
            'token'         => $token,
            'tipe'          => 'Bearer',
            'berlaku_detik' => $jwt_berlaku,
            'user'          => ['username' => $akun['username'], 'role' => $akun['role']]
        ]);
    }

    // ================================================================
    // <-- BARU P7: SEMUA alamat di bawah ini WAJIB membawa token yang sah.
    // wajib_login() mengembalikan data pengguna, atau langsung 401 dan berhenti.
    // ================================================================
    elseif ($resource != '') {
        $pengguna = wajib_login();
    }

    // ================================================================
    // LANGKAH 2: ROUTING  -> /me   (BARU P7)
    // ================================================================
    if ($resource == 'me' && $method == 'GET') {
        kirim(['status' => 'sukses', 'data' => [
            'id' => $pengguna['sub'], 'username' => $pengguna['username'],
            'role' => $pengguna['role'], 'token_habis_pada' => date('Y-m-d H:i:s', $pengguna['exp'])
        ]]);
    }

    // ================================================================
    // ROUTING  -> /mhs
    // ================================================================
```

```
elseif ($resource == 'mhs') {

    // ---------- ID harus angka (jika diisi) ----------
    if ($id !== null && !ctype_digit($id)) {
        kirim(['status' => 'error', 'pesan' => 'ID harus berupa angka'], 400);
    }

    // ---------- READ: GET /mhs  (semua data) ----------
    if ($method == 'GET' && $id === null) {
        $hasil = $db->query("SELECT * FROM mhs ORDER BY id")->fetchAll();
        kirim(['status' => 'sukses', 'data' => $hasil]);
    }

    // ---------- READ: GET /mhs/3  (satu data) ----------
    elseif ($method == 'GET') {
        $stmt = $db->prepare("SELECT * FROM mhs WHERE id = ?");
        $stmt->execute([$id]);
        $baris = $stmt->fetch();                  // false jika tidak ada
        if (!$baris) {
            kirim(['status' => 'error', 'pesan' => 'Mahasiswa tidak ditemukan'], 404);
        }
        kirim(['status' => 'sukses', 'data' => $baris]);
    }

    // ---------- CREATE: POST /mhs ----------
    elseif ($method == 'POST' && $id === null) {
        wajib_admin($pengguna);                   // <-- BARU P7: OTORISASI, hanya admin
        cek_input($input);                        // data wajib lengkap, jika tidak -> berhenti (400)

        // NIM tidak boleh kembar
        $stmt = $db->prepare("SELECT id FROM mhs WHERE nim = ?");
        $stmt->execute([$input['nim']]);
        if ($stmt->fetch()) {
            kirim(['status' => 'error', 'pesan' => 'NIM sudah terdaftar'], 409);
        }

        $stmt = $db->prepare("INSERT INTO mhs (nim, name, major) VALUES (?, ?, ?)");
        $stmt->execute([$input['nim'], $input['name'], $input['major']]);

        $idBaru = $db->lastInsertId();            // id yang baru dibuat MySQL
        kirim(['status' => 'sukses', 'pesan' => 'Data ditambahkan', 'id' => (int)$idBaru], 201);
    }

    // ---------- UPDATE: PUT /mhs/3 ----------
    elseif ($method == 'PUT' && $id !== null) {
        wajib_admin($pengguna);                   // <-- BARU P7
        cek_input($input);

        // Pastikan datanya ada dulu
        $stmt = $db->prepare("SELECT id FROM mhs WHERE id = ?");
        $stmt->execute([$id]);
        if (!$stmt->fetch()) {
            kirim(['status' => 'error', 'pesan' => 'Mahasiswa tidak ditemukan'], 404);
        }

        // NIM baru tidak boleh dipakai mahasiswa LAIN (id <> ?)
        $stmt = $db->prepare("SELECT id FROM mhs WHERE nim = ? AND id <> ?");
        $stmt->execute([$input['nim'], $id]);
        if ($stmt->fetch()) {
            kirim(['status' => 'error', 'pesan' => 'NIM sudah dipakai mahasiswa lain'], 409);
        }

        $stmt = $db->prepare("UPDATE mhs SET nim = ?, name = ?, major = ? WHERE id = ?");
        $stmt->execute([$input['nim'], $input['name'], $input['major'], $id]);
        kirim(['status' => 'sukses', 'pesan' => 'Data diubah']);
    }

    // ---------- DELETE: DELETE /mhs/3 ----------
    elseif ($method == 'DELETE' && $id !== null) {
        wajib_admin($pengguna);                   // <-- BARU P7
        $stmt = $db->prepare("DELETE FROM mhs WHERE id = ?");
        $stmt->execute([$id]);
        if ($stmt->rowCount() == 0) {             // tidak ada baris yang terhapus
            kirim(['status' => 'error', 'pesan' => 'Mahasiswa tidak ditemukan'], 404);
        }
        kirim(['status' => 'sukses', 'pesan' => 'Data dihapus']);
    }

    // ---------- Kombinasi lain (mis. DELETE /mhs tanpa id) ----------
    else {
```

```
            kirim(['status' => 'error', 'pesan' => 'Method tidak diizinkan'], 405);
        }
    }

    // ================================================================
    // ROUTING  -> /log   (melihat catatan log dari database)
    // ================================================================
    elseif ($resource == 'log' && $method == 'GET') {
        wajib_admin($pengguna);                      // <-- BARU P7: log hanya boleh dilihat admin
        $hasil = $db->query("SELECT * FROM log_api ORDER BY id DESC LIMIT 20")->fetchAll();
        // Perhatian: kirim() juga mencatat request /log ini ke log_api.
        kirim(['status' => 'sukses', 'data' => $hasil]);
    }

    // ================================================================
    // ROUTING  -> alamat kosong  (cek server hidup)
    // ================================================================
    elseif ($resource == '') {
        kirim(['status' => 'sukses', 'pesan' => 'API Mahasiswa aktif']);
    }

    // Alamat lain yang tidak dikenal
    else {
        kirim(['status' => 'error', 'pesan' => 'Alamat tidak ditemukan'], 404);
    }

} catch (PDOException $e) {
    // Error database tak terduga. Detailnya TIDAK dikirim ke client (bisa membocorkan struktur tabel).
    kirim(['status' => 'error', 'pesan' => 'Terjadi kesalahan pada database'], 500);
}



// ================================================================
// FUNGSI: cek_input()  -- memastikan data dari client lengkap
// Jika ada yang kurang, langsung kirim error 400 dan berhenti.
// ================================================================
function cek_input($input)
{
    // Jika body bukan JSON yang benar, $input bukan array.
    if (!is_array($input)) {
        kirim(['status' => 'error', 'pesan' => 'Kirim data dalam format JSON'], 400);
    }

    // Tiga field ini wajib ada dan tidak boleh kosong.
    foreach (['nim', 'name', 'major'] as $field) {
        if (empty($input[$field])) {
            kirim(['status' => 'error', 'pesan' => "Field $field wajib diisi"], 422);
        }
    }

    // NIM harus angka saja.
    if (!ctype_digit((string)$input['nim'])) {
        kirim(['status' => 'error', 'pesan' => 'NIM harus berupa angka'], 422);
    }
}
```

| Request | Apa yang terjadi di kode |
|---|---|
| POST /login | validasi input (422) → cek batas gagal login (429) → cari user → `password_verify` → 401 atau token |
| GET /mhs tanpa token | `wajib_login()` → `cek_token()` null → 401 + header `WWW-Authenticate` |
| POST /mhs oleh budi | lolos `wajib_login()` → `wajib_admin()` melihat role `user` → 403 |
| POST /mhs oleh admin | lolos keduanya → validasi data → INSERT → 201 |

### Mengapa pesan login salah dibuat sama?

Untuk "username tidak ada" dan "password salah" server membalas pesan yang **sama**: *Username atau password salah*. Jika berbeda, penyerang dapat menebak username mana yang terdaftar (**user enumeration**).

## LANGKAH 8 .htaccess — meneruskan header Authorization

Apache dalam beberapa konfigurasi membuang header `Authorization` sebelum sampai ke PHP, sehingga server selalu menjawab 401 walaupun token benar. Dua baris pertama di bawah mencegahnya.

**server/.htaccess**

```
# ======================================================================
# .htaccess  --  Routing sederhana
# Semua alamat (mis. /mhs/3) dialihkan ke index.php supaya URL tampak bersih.
# Syarat: mod_rewrite aktif dan AllowOverride All (XAMPP/Laragon sudah default).
# ======================================================================
RewriteEngine On

# <-- BARU P7: Apache sering MEMBUANG header Authorization sebelum sampai ke PHP.
# Dua baris ini menyalinnya agar token JWT terbaca.
RewriteCond %{HTTP:Authorization} .
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]

# Jika yang diminta BUKAN file asli dan BUKAN folder asli -> serahkan ke index.php
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [L]
```

# 6. Langkah 9–10: Pengujian

## LANGKAH 9 Uji dengan Postman

1. Buat **Environment** dengan variabel `base` = `http://localhost/api-mhs-auth/server` dan `token` (kosong).
2. Kirim `POST {{base}}/login`, Body raw JSON: `{"username":"budi","password":"budi123"}`. Salin nilai `token` dari jawaban.
3. Pada request lain: tab **Authorization** → Type **Bearer Token** → isi `{{token}}` (atau tab Headers: `Authorization: Bearer <token>`).
4. Jalankan skenario berikut, catat hasilnya di lembar observasi.

| No | Method | URL | Kondisi | Harapan |
|---|---|---|---|---|
| 1 | GET | /mhs | tanpa token | 401 |
| 2 | POST | /login | password salah | 401 |
| 3 | POST | /login | tanpa field password | 422 |
| 4 | POST | /login | admin / admin123 | 200 + token |
| 5 | GET | /me | token budi | 200, role user |
| 6 | GET | /mhs | token budi | 200 |
| 7 | POST | /mhs | token budi | **403** |
| 8 | POST | /mhs | token admin, data lengkap | 201 |
| 9 | DELETE | /mhs/{id} | token admin | 200 |
| 10 | GET | /log | token budi / token admin | 403 / 200 |
| 11 | GET | /mhs | token diubah 1 huruf | 401 |
| 12 | POST | /login | 6x salah berturut-turut (username sama) | 429 mulai ke-6 |

> **Eksperimen masa berlaku**
>
> Ubah `$jwt_berlaku` menjadi `30`, login, lalu panggil `GET /mhs` sebelum dan sesudah 30 detik. Pada detik ke-31 server membalas **401** karena token kedaluwarsa. Kembalikan ke 3600 setelah selesai.

## LANGKAH 10 Uji otomatis dari sisi client — test_client.php

Skrip login sebagai admin dan budi, lalu memeriksa 28 kondisi: token sah/palsu, otorisasi per role, CRUD, SQL Injection pada login, rate limiting, dan log. Bagian fungsi inti:

client/test_client.php (fungsi inti)

```
function panggil($method, $path, $data = null, $token = null)
{
    global $base;

    $header = [];
    if ($data !== null) {
        $header[] = 'Content-Type: application/json';
    }
    if ($token !== null) {
        $header[] = 'Authorization: Bearer ' . $token;      // <-- cara membawa token
    }

    $ch = curl_init($base . $path);
    curl_setopt($ch, CURLOPT_CUSTOMREQUEST, $method);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_TIMEOUT, 10);
    curl_setopt($ch, CURLOPT_HTTPHEADER, $header);
    if ($data !== null) {
        curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($data));
    }

    $isi    = curl_exec($ch);
```

```
    $status = curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
    curl_close($ch);

    return [$status, json_decode($isi, true)];
}
```

**Terminal**

```
php client/test_client.php http://localhost/api-mhs-auth/server
```

**Contoh keluaran test_client.php**

```
1. AUTENTIKASI: siapa kamu?
  [PASS] GET /mhs tanpa token -> 401
  [PASS] Login password salah -> 401
  [PASS] Login username tidak ada -> 401 (pesan sama, tidak membocorkan)
  [PASS] Login tanpa password -> 422
  [PASS] Login dengan SQL Injection ' OR '1'='1 -> 401 (gagal menembus)
  [PASS] Login admin benar -> dapat token
  [PASS] Login budi benar -> dapat token
   token berbentuk 3 bagian dipisah titik: eyJhbGciOiJIUzI1NiIsInR5cCI6Ik...
  [PASS] Token terdiri dari 3 bagian (header.payload.signature)
  [PASS] GET /me dengan token -> 200, username budi

2. TOKEN PALSU / RUSAK harus ditolak
  [PASS] Token asal-asalan -> 401
  [PASS] Tanda tangan diubah -> 401
  [PASS] Payload diubah jadi admin (tanpa kunci) -> 401

3. OTORISASI: bolehkah kamu?  (budi = role user)
  [PASS] user: GET /mhs -> 200 (boleh melihat)
  [PASS] user: POST /mhs -> 403 (tidak boleh menambah)
  [PASS] user: DELETE /mhs/1 -> 403
  [PASS] user: GET /log -> 403

4. ADMIN boleh melakukan CRUD
  [PASS] admin: POST /mhs -> 201
  [PASS] admin: POST NIM kembar -> 409
  [PASS] admin: POST data tidak valid -> 422
  [PASS] admin: PUT -> 200
  [PASS] admin: id "1 OR 1=1" -> 400 (validasi input)
  [PASS] admin: DELETE -> 200

5. RATE LIMITING: pembatasan percobaan login
   status 7 percobaan berturut-turut: 401, 401, 401, 401, 401, 429, 429
  [PASS] 5 percobaan pertama ditolak 401
  [PASS] Percobaan ke-6 dst diblokir 429 (Too Many Requests)
  [PASS] Akun asli (admin) tidak ikut terkunci
  ...
  [PASS] Log mencatat percobaan ditolak (401 / 403 / 429)

HASIL: 28 dari 28 pengujian lulus
```

**Jika muncul pesan "terlalu banyak request"**

`$batas_umum` = 120 request/menit per IP, sedangkan satu kali tes memakai sekitar 60 request. Menjalankan tes lebih dari dua kali dalam satu menit akan terkena **429**. Tunggu 1 menit.

# 7. Langkah 11: Client Web dengan Login

## LANGKAH 11 index.html — login, token, dan role

Setelah login, token disimpan di variabel JavaScript `token`. Fungsi `kirim()` memasangnya di header `Authorization` untuk **semua** request berikutnya, sehingga halaman lain tidak perlu mengurus token.

**client/index.html (login dan pengiriman token)**

```
let token = null;

function alamat() { return document.getElementById('alamat').value; }
function tampilPesan(teks) { document.getElementById('pesan').textContent = teks; }

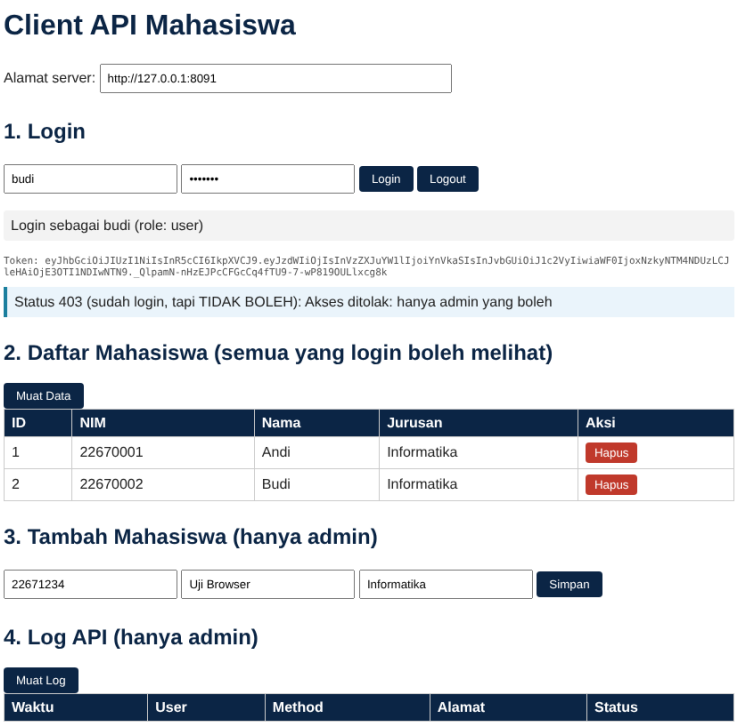
// ---------------------------------------------------------------
// kirim() : satu-satunya fungsi yang memanggil API.
// Jika sudah login, token otomatis dipasang di header Authorization.
// ---------------------------------------------------------------
async function kirim(method, path, data) {
  const header = { 'Content-Type': 'application/json' };
  if (token) {
    header['Authorization'] = 'Bearer ' + token;      // <-- cara membawa token
  }
  const respons = await fetch(alamat() + path, {
    method: method,
    headers: header,
    body: data ? JSON.stringify(data) : undefined
  });
  const hasil = await respons.json();
  return { status: respons.status, hasil: hasil };
}

// Penjelasan status code untuk mahasiswa
function artiStatus(status, pesan) {
  if (status == 401) return 'Status 401 (belum login / token tidak sah): ' + pesan;
  if (status == 403) return 'Status 403 (sudah login, tapi TIDAK BOLEH): ' + pesan;
  if (status == 429) return 'Status 429 (terlalu banyak request): ' + pesan;
  return 'Status ' + status + ': ' + pesan;
}

// ---------------------------------------------------------------
// LOGIN : kirim username + password, simpan token yang diterima
// ---------------------------------------------------------------
async function login() {
  const r = await kirim('POST', '/login', {
    username: document.getElementById('username').value,
    password: document.getElementById('password').value
  });
  if (r.status == 200) {
    token = r.hasil.token;
    document.getElementById('status').textContent =
      'Login sebagai ' + r.hasil.user.username + ' (role: ' + r.hasil.user.role + ')';
    document.getElementById('tokenTampil').textContent = 'Token: ' + token;
    tampilPesan('Login berhasil. Token tersimpan dan akan dikirim di setiap request.');
  } else {
    tampilPesan(artiStatus(r.status, r.hasil.pesan));
  }
}

function logout() {
  token = null;                                       // membuang token = logout
  document.getElementById('status').textContent = 'Belum login.';
  document.getElementById('tokenTampil').textContent = '';
  tampilPesan('Anda sudah logout.');
}
```

1. Buka `client/index.html`, pastikan alamat server `http://localhost/api-mhs-auth/server`.
2. Tanpa login, klik **Muat Data** → pesan 401. Login sebagai **budi**, muat data, lalu coba **Simpan** → pesan **403**.
3. Login sebagai **admin** → tambah data berhasil; klik **Muat Log** dan lihat kolom **User**.




*Gambar 1. Login sebagai budi (role user): melihat data boleh, menambah ditolak 403.*

**Di mana token disimpan di browser?**

Modul ini menyimpannya di variabel agar sederhana (hilang saat halaman dimuat ulang). `localStorage` praktis tetapi dapat dibaca skrip jahat bila halaman terkena XSS. Pilihan penyimpanan token adalah bahan diskusi keamanan di kelas.

# 8. Langkah 12: Rate Limiting dan Validasi Input

LANGKAH 12 **Membatasi request dan memeriksa input**

## 12a. Rate limiting login (brute force)

Penyerang dapat mencoba ribuan password. Server menghitung login gagal (status 401) untuk **kombinasi IP + username** dalam 1 menit memakai tabel `log_api`. Lebih dari `$batas_login` (5) → **429 Too Many Requests** + header `Retry-After`.

1. Di Postman, kirim `POST /login` dengan password salah berulang kali untuk username `budi`.
2. Percobaan ke-6 mendapat **429**. Periksa dengan SQL:

**phpMyAdmin → SQL**

```
SELECT waktu, username, status, pesan FROM log_api
WHERE path LIKE '%/login' ORDER BY id DESC LIMIT 10;
```

3. Tunggu 1 menit, login benar → berhasil kembali.

## 12b. Rate limiting umum

**Terminal**

```
php latihan/uji_rate_limit.php http://localhost/api-mhs-auth/server
```

**Contoh keluaran**

```
Mengirim 150 request cepat ke http://localhost/api-mhs-auth/server
  status 200 : 120 kali
  status 429 : 30 kali
Jika ada status 429, rate limiting bekerja.
```

**fungsi.php — batasi_request()**

```
// batasi_request() : RATE LIMITING umum per IP
//   Menghitung jumlah request dari IP ini dalam 1 menit terakhir (dari tabel log_api).
//   Jika sudah terlalu banyak -> 429 Too Many Requests.
// ---------------------------------------------------------------------
function batasi_request()
{
    global $db, $batas_umum;

    $semenit_lalu = date('Y-m-d H:i:s', time() - 60);
    $stmt = $db->prepare("SELECT COUNT(*) FROM log_api WHERE ip = ? AND waktu >= ?");
    $stmt->execute([$_SERVER['REMOTE_ADDR'], $semenit_lalu]);
    $jumlah = $stmt->fetchColumn();

    if ($jumlah >= $batas_umum) {
        header('Retry-After: 60');           // beri tahu client: coba lagi 60 detik lagi
        kirim(['status' => 'error', 'pesan' => 'Terlalu banyak request. Coba lagi 1 menit lagi'], 429);
    }
}
```

## 12c. Validasi input

| Serangan / kesalahan | Pertahanan di kode | Hasil |
|---|---|---|
| Login tanpa password / bukan teks | `empty()` dan `is_string()` | 422 |
| `' OR '1'='1` pada username | prepared statement `WHERE username = ?` | 401 (tidak menembus) |
| id `1 OR 1=1` pada URL | `ctype_digit($id)` | 400 |
| NIM bukan angka | `ctype_digit()` di `cek_input()` | 422 |
| Body bukan JSON | `is_array($input)` | 400 |

| Praktik keamanan dasar | Pada proyek ini |
|---|---|
| Jangan simpan password asli | `password_hash()` / `password_verify()` |
| Jangan bocorkan informasi | Pesan login seragam; detail error database tidak dikirim ke client |
| Prinsip least privilege | Role admin vs user; log hanya admin |
| Batasi percobaan | Rate limiting login dan umum |
| Catat jejak | `log_api` memuat IP, username, status |
| Gunakan HTTPS | Wajib di produksi: token dan password bisa disadap pada HTTP biasa |

# 9. Bacaan: Pengantar OAuth2

OAuth 2.0 bukan cara login, melainkan **kerangka otorisasi pihak ketiga**: aplikasi (client) mendapat **izin terbatas** ke data pengguna di layanan lain **tanpa mengetahui password pengguna**. Contohnya tombol "Login dengan Google" atau aplikasi yang membaca kalender Anda.

| Peran | Contoh |
|---|---|
| Resource Owner | Pengguna pemilik data (Anda). |
| Client | Aplikasi yang meminta akses (mis. aplikasi akademik kampus). |
| Authorization Server | Yang memberi izin dan menerbitkan token (mis. akun Google). |
| Resource Server | API yang menyimpan data dan memeriksa token. |

**Alur Authorization Code (disederhanakan)**

```
1. Aplikasi mengarahkan pengguna ke Authorization Server ("Izinkan aplikasi X membaca profil Anda?")
2. Pengguna login DI SITUS PENYEDIA dan menyetujui
3. Authorization Server memberi KODE ke aplikasi
4. Aplikasi menukar kode dengan ACCESS TOKEN
5. Aplikasi memanggil API (Resource Server) dengan  Authorization: Bearer <access token>
```

> **Kaitan dengan praktikum**
>
> Header `Authorization: Bearer ...` yang kita pakai **sama** dengan OAuth2. Bedanya, pada praktikum ini API kita sendiri yang memeriksa password dan menerbitkan token. Pada OAuth2 penerbit token adalah pihak lain, dan token sering berupa JWT. Memahami JWT berarti sudah memahami setengah OAuth2.

# 10. Pemecahan Masalah

| Gejala | Penyebab | Solusi |
|---|---|---|
| Selalu 401 padahal token benar | Apache membuang header Authorization | Pastikan `.htaccess` versi P7 dipakai; mod_rewrite aktif. |
| 401 setelah beberapa saat | Token kedaluwarsa (`exp`) | Login ulang. |
| Semua token lama tiba-tiba 401 | `$jwt_secret` diubah | Login ulang. |
| Login admin selalu 401 | Tabel users belum/ salah diimpor | Impor ulang `mhs.sql` versi P7. |
| Error 500 saat membuka /mhs | Kolom `username` belum ada di `log_api` | Impor ulang `mhs.sql` versi P7. |
| 429 padahal baru sekali mencoba | Batas request/menit terlewati (mis. habis tes) | Tunggu 1 menit. |
| Client web: gagal terhubung saat memakai token | Header Authorization belum diizinkan CORS | Cek `Access-Control-Allow-Headers` di index.php. |
| `Bearer` salah ketik | Format header harus `Authorization: Bearer <token>` | Periksa spasi dan huruf. |

# 11. Lembar Observasi

| No | Skenario | Harapan | Hasil | Catatan / baris log |
|---|---|---|---|---|
| 1 | GET /mhs tanpa token | 401 | | |
| 2 | Login password salah | 401 | | |
| 3 | Login admin benar | 200 | | |
| 4 | budi: GET /mhs | 200 | | |
| 5 | budi: POST /mhs | 403 | | |
| 6 | admin: POST /mhs | 201 | | |
| 7 | budi: GET /log | 403 | | |
| 8 | Token diubah 1 huruf | 401 | | |
| 9 | Token kedaluwarsa (exp 30 detik) | 401 | | |
| 10 | 6x login salah | 429 | | |
| 11 | test_client.php | 28/28 | | |

**Pertanyaan refleksi:**

1. Jelaskan perbedaan 401 dan 403 dengan contoh dari praktikum ini.
2. Mengapa payload JWT tidak boleh berisi password, padahal token "ditandatangani"?
3. Apa yang terjadi bila penyerang mengubah `"role":"user"` menjadi `"admin"` di payload? Mengapa gagal?
4. Mengapa password disimpan sebagai hash dan bukan dienkripsi?
5. Apa risiko jika token disimpan di localStorage? Apa kompromi yang ada?
6. Apa beda JWT yang kita buat sendiri dengan OAuth2?

# 12. Tugas Mahasiswa

## Tugas Pertemuan 7 — Pengembangan autentikasi

Kerjakan pada salinan proyek `api-mhs-auth`:

1. **Register:** tambahkan `POST /register` (publik): username unik, password minimal 8 karakter, disimpan dengan `password_hash`, role otomatis `user`. Tanggapi 201 / 409 / 422.
2. **Ganti password:** tambahkan `PUT /me/password` (perlu token): wajib mengirim password lama yang benar.
3. **Role baru:** tambahkan role `dosen` yang boleh PUT (ubah) tetapi tidak boleh DELETE. Perluas fungsi otorisasi menjadi `wajib_role(["admin","dosen"])`.
4. **Analisis log:** tulis 3 query SQL pada `log_api`: jumlah login gagal per username, 5 IP dengan request terbanyak, dan jumlah 401/403/429 per hari.
5. **Uji:** perbarui `test_client.php` dengan minimal 8 pengecekan baru untuk fitur Anda.

## Tantangan tambahan

- Tambahkan klaim `jti` (ID unik token) dan tabel `token_blacklist` agar **logout** benar-benar membatalkan token di server.
- Buat dua token: access token (5 menit) dan refresh token (1 hari), dan endpoint `POST /refresh`.
- Kunci akun sementara setelah 10 gagal login dalam 10 menit, dan catat di log.
- Tuliskan analisis ancaman singkat: 3 cara token bisa dicuri dan pertahanannya.

| Aspek | Bobot | Indikator |
|---|---|---|
| Autentikasi (login, hash, JWT) | 30% | Register + login berfungsi; password ter-hash; token benar. |
| Otorisasi | 20% | Role diterapkan benar; 401 vs 403 tepat. |
| Keamanan dasar | 20% | Validasi input, prepared statement, rate limiting. |
| Pengujian client | 20% | Skrip tes baru, hasil PASS. |
| Kode dan laporan | 10% | Komentar, analisis log. |

## 13. Referensi

- RFC 7519 — JSON Web Token (JWT): https://www.rfc-editor.org/rfc/rfc7519
- RFC 6749 — The OAuth 2.0 Authorization Framework: https://www.rfc-editor.org/rfc/rfc6749
- RFC 6750 — Bearer Token Usage: https://www.rfc-editor.org/rfc/rfc6750
- OWASP — Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP — Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP API Security Top 10: https://owasp.org/API-Security/
- PHP Manual — password_hash: https://www.php.net/manual/en/function.password-hash.php
- jwt.io — Introduction to JSON Web Tokens: https://jwt.io/introduction